Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 15

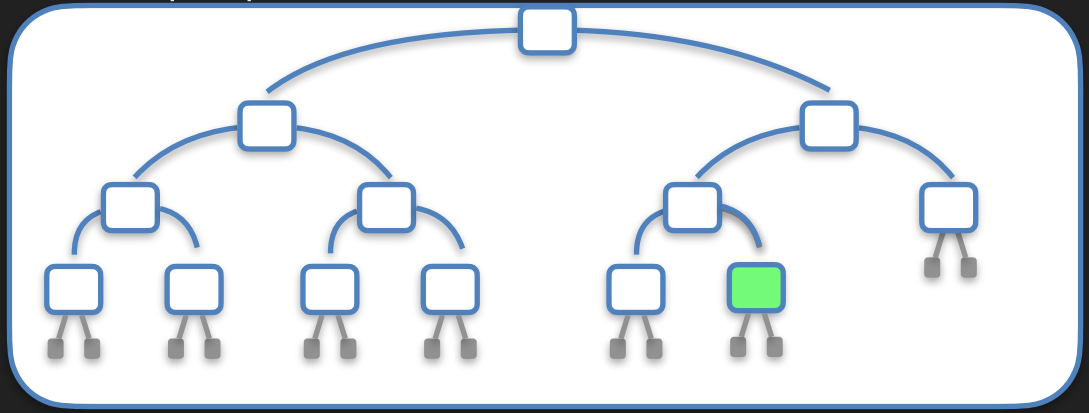
Heaps

BST

- Time for searching, inserting and deleting from a BST is all O(h), where h is the height of the BST
- Recall: the number of internal nodes a binary tree of height h can have is $n \leq n_{max} = 2^0 + 2^1 + \ldots + 2^h = 2^{h+1} - 1$
  - So $h \geq \log_2(n+1) - 1$. Ideally, we should keep h = O(log n), where n is the number of internal nodes
- But without additional balancing mechanisms, h can grow well beyond Θ(log n)
  - Demo

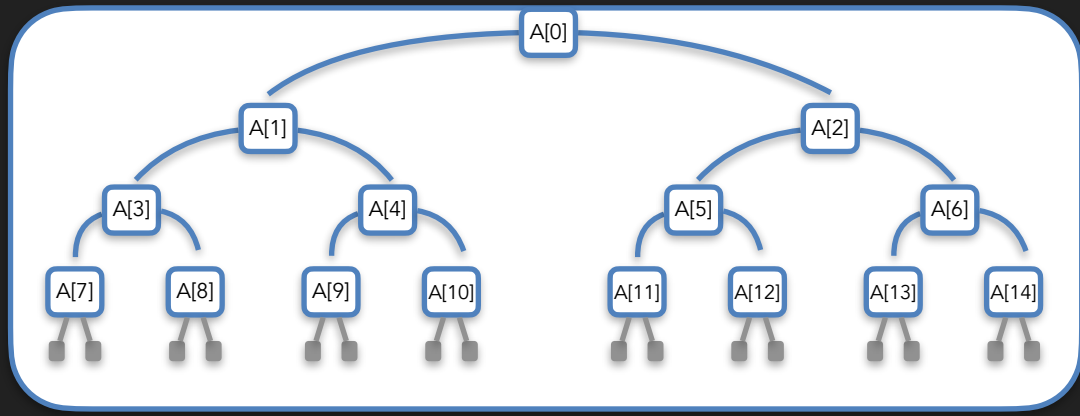
Nearly Complete Binary Tree

- In a nearly complete binary tree, only last level can be incomplete, and there all the internal nodes are to the left of all the leaves (if any)
  - $2^h \leq n < 2^{h+1}$ (recall convention: h=0 for n=1)
- If we didn't need to maintain the BST property, easy to maintain a nearly complete binary tree
  - Always insert and delete at the "bottom-most, right-most" position
    - To delete another node, first copy the bottom-right element to that node, and then remove the bottom-right node



Nearly Complete Binary Tree in an Array

- Can maintain a nearly complete binary tree without using any pointers, in a vector!
- Idea: Store the nodes (just the data) in a vector in "breadth-first order"
  - Contrast with "depth-first" order of pre/in/post-order traversal, where the left-subtree is finished before the right subtree is started



- `A[0]` is root. Children of `A[i]` are `A[2i+1]` and `A[2i+2]` (right/both possibly leaves)
  - Why? Verify base case of i=0. Now, if `A[i-1]` has children `A[j]`, `A[j+1]`, then `A[i]` has `A[j+2]`, `A[j+3]`. Inductively, `j=2i-1`.
- Supports insertion/deletion at the back of the vector

Attendance Break!

Login using CSE LDAP id at

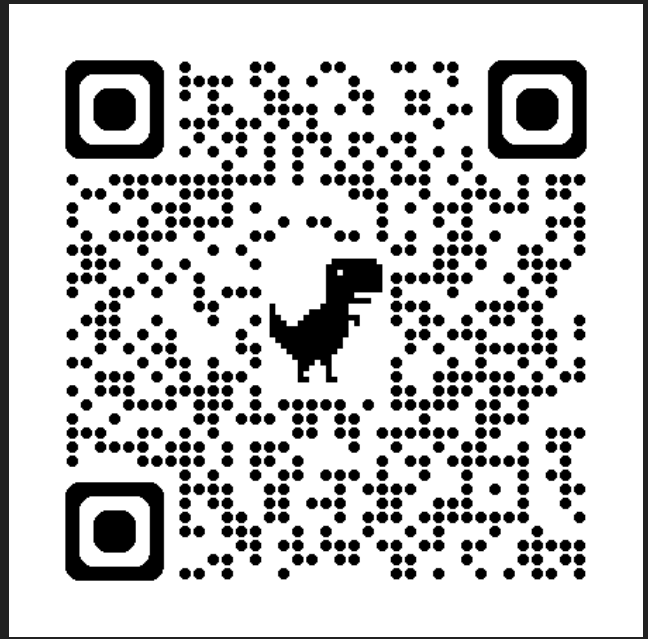
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



Containers in C++ STL

- So far:
  - `std::vector` : Dynamically resized array
  - `std::list` : Doubly linked list (`std::forward_list`: singly-linked list)
  - `std::queue, std::stack, std::deque` : Deque has random access
  - `std::set, std::map, std::multiset, std::multimap`
    - Balanced Binary Search Tree (balancing covered later)
- Coming up later
  - `std::unordered_{set,map,multiset,multimap}`: Hash table
- Today:
  - `std::priority_queue` : Heap

Priority Queue

- Stack implemented LIFO, Queue implemented FIFO
  - Retrieval order irrespective of the actual content of the items
- But often we would like to assign items priorities as they are entered into a queue, and retrieve the highest priority item first
  - E.g., when urgent tasks should be processed first, not depending on how long they have been in the queue
  - Or, as a subroutine in many algorithms, where the "best" option so far should be explored first

Maintaining the Minimum

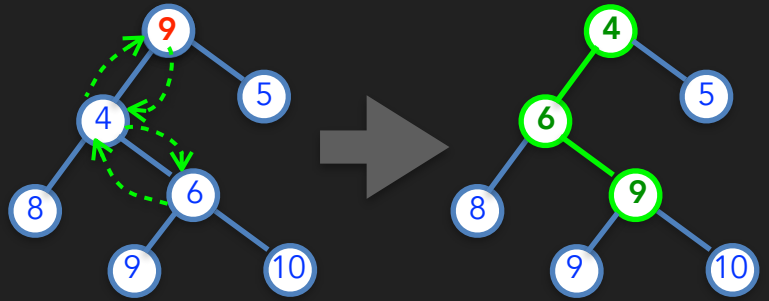
- Given a sequence of numbers, finding the minimum is easy
  - In O(n) time, with O(1) extra space
- How about finding the minimum, removing it, and finding the next minimum and so forth?
  - Equivalent to sorting! (Either task enables the other with little overhead)
  - Sorting n numbers takes Θ(n log n) time [later]. So best possible worst-case (or even amortised) bound is O(log n)
- Can we *repeatedly* find and remove the minimum in O(log n) *worst case* time?
  - Yes! Using a binary heap.

A Binary Heap

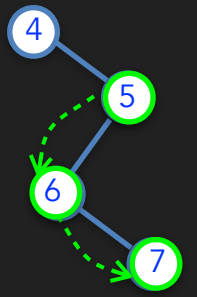
- A nearly-complete binary tree with the heap property:
  - A child is at least as big as its parent
- In every subtree, root is the minimum
- How do you insert/delete a node in O(log n) time?
  - Remember: to maintain the nearly-complete binary tree structure, we should add/remove at the bottom-right
  - To insert, add a node at the bottom-right and then update its value maintaining the heap property
  - To delete, remove the bottom-right entry and update the value of the node to be deleted to be the removed entry
- Subroutine needed: Update the value in a node of the heap while maintaining the heap property, in O(log n) time

Changing a Single Node's Key

- Can bubble down or bubble up the changed node to fix the heap property
  - If update decreases the value of the node, bubble up.
    Else bubble down.
- Bubbling up algorithm:
  - Compare new key with parent; if not in heap order, swap; repeat until heap property fixed.
- Bubbling down algorithm:
  - Compare new key with both children; if not in heap order, swap with the smaller child; repeat until heap property fixed.
- Why does this work?



Changing a Single Node's Key



- Equivalent version: Instead of swapping, copy from next active node to previously active node. Insert new value only after finding its final position.
- Invariant: After each iteration, we have a valid heap
  - Initially true
  - If bubbling up: Active node's parent ≤ active node's children
  - If bubbling down: child copied up is the smallest among the active node and its children.
- On stopping, new key can be placed at the active node keeping heap valid
- Will stop in at most height many steps

Changing a Single Node's Key

```
void update(node* actv, int newkey) {
  if(actv->key > newkey) bubbleUp(actv,newkey); else bubbleDown(actv,newkey);
}
void bubbleUp(node* actv, int newkey) {
  for(; actv->parent && actv->parent->key > newkey; actv = actv->parent;)
    actv->key = actv->parent->key;
  actv->key = newkey; //outside the loop: newkey copied into actv only now
}
void bubbleDown(node* actv, int newkey) {
  for(bool done=false; !done; ) {
    int smallest = newkey;
    if(actv->left && actv->left->key < smallest) smallest = actv->left->key;
    if(actv->right && actv->right->key < smallest) smallest = actv->right->key;
    if(smallest==newkey) done = true;
    else { // bubble the smaller child up (left child guaranteed to exist)
      actv->key = smallest;
      actv = (smallest==actv->left->key) ? actv->left : actv->right;
    }
  }
  actv->key = newkey; //outside the loop: newkey copied into actv only now
}
```